

K-GUARD

Dossier de conception technique

Version 1.7.0 — Native Security Monitoring & Grouped Network Policies

Plateforme DevSecOps / SecOps Cloud-Native pour clusters Kubernetes K3s

Objectif	Centraliser la visibilité opérationnelle, la sécurité runtime et la gouvernance réseau d'un cluster K3s.
Approche	Security by Design, backend-mediated integrations, lecture seule pour Wazuh, opérations Sentinel explicitement confirmées.
Statut	Proof of Concept / MVP personnel en évolution continue. Non présenté comme une plateforme de production certifiée.
Auteur	Kamal Guidadou
Date	10 août 2026

Document de référence pour la version actuelle du projet. Les anciennes versions restent utiles comme historique d'architecture, mais ce document décrit spécifiquement l'état v1.7.0.

Sommaire. Structure du document

Réf.	Section
01	Vision et périmètre
02	Architecture globale
03	System Overview & exploitation K3s
04	Security Runtime — Falco & K-Guard AI
05	Endpoint & Compliance — Wazuh
06	Network Sentinel & gouvernance réseau
07	Sécurité applicative et modèle d'accès
08	Intégrations et notifications
09	Déploiement, observabilité et maintenance
10	Validation v1.7.0 et limites
11	Conclusion et perspectives

Le document privilégie une lecture d'ingénierie : architecture, responsabilités des composants, contrôles de sécurité, flux de données et limites connues.

1. Vision et périmètre

K-Guard est une plateforme personnelle DevSecOps / SecOps conçue pour fournir une vue opérationnelle unifiée d'un cluster K3s. La version 1.7.0 consolide trois axes : supervision de l'infrastructure, visibilité de sécurité runtime et gouvernance réseau.

Le projet est volontairement présenté comme un MVP expérimental. Il vise à démontrer une capacité de conception, d'intégration et d'exploitation autour de Kubernetes, plutôt qu'à se substituer à un SOC, un programme de gestion des vulnérabilités ou une plateforme de sécurité d'entreprise. Cette posture est explicitement conservée dans la documentation actuelle du projet. ■ ■ ■

La version 1.7.0 introduit surtout une gouvernance plus contrôlée de NetworkPolicy : sélection de groupes, ciblage de namespaces, plan de durcissement en lecture seule, confirmation explicite et suppression limitée aux ressources gérées par Sentinel. ■ ■ ■

Principes directeurs

- **Security by Design** : réduire l'exposition des secrets et encadrer les opérations sensibles dès l'architecture.
- **Backend-mediated security** : le navigateur ne communique pas directement avec les services protégés.
- **Read-only by default** pour l'intégration Wazuh exposée dans K-Guard.
- **Progressive Zero-Trust** : segmenter progressivement les flux plutôt que déployer des restrictions aveugles.
- **Operational clarity** : fournir des informations directement exploitables depuis une interface native.

Périmètre fonctionnel v1.7.0

Domaine	Responsabilité
Infrastructure	État des workloads, métriques CPU/RAM, logs de pods, capacité nœud, stockage et informations K3s.
Runtime Security	Réception et visualisation des événements Falco, enrichissement K-Guard AI et persistance locale des événements.
Endpoint Security	Inventaire et visibilité des alertes/posture Wazuh via le backend, sans administration Wazuh depuis l'UI.
Network Governance	Topologie, posture, recommandations et gestion groupée de NetworkPolicies via Sentinel.
ChatOps	Notifications opérationnelles et sécurité vers Cisco Webex.
Administration	Authentification JWT, configuration des intégrations, documentation API et maintenance contrôlée.

2. Architecture globale

K-Guard adopte une architecture médiée par le backend. Le frontend Vue 3 / TypeScript communique avec FastAPI au moyen de requêtes authentifiées. Les accès aux services Kubernetes et aux intégrations de sécurité restent côté backend. ■ ■ ■

Vue d'ensemble des flux

Vue Web K-Guard Vue 3 / TypeScript	HTTPS / JWT	Backend K-Guard FastAPI / Python
System Overview Security Runtime Wazuh Sentinel Settings		Kubernetes API SQLite Security integrations

- **Kubernetes / K3s** : source d'état des workloads, métriques, logs et ressources réseau.
- **Falco** : détection runtime au niveau comportemental. K-Guard ne lui substitue pas un SIEM.
- **Fluent Bit** : acheminement des événements runtime vers la chaîne K-Guard.
- **K-Guard AI** : enrichissement et contextualisation des événements Falco avant leur présentation.
- **Wazuh** : source distincte pour l'inventaire endpoint et les données de sécurité endpoint.
- **Sentinel** : moteur de planification, audit et opérations contrôlées autour des NetworkPolicies.

Séparation des responsabilités de sécurité

Composant	Rôle	Ce qu'il ne faut pas lui attribuer
Falco	Runtime Security : détecter des comportements suspects dans les workloads.	Inventaire endpoint / SIEM global.
Wazuh	SIEM/XDR endpoint : inventaire, alertes et contexte de sécurité endpoint exposés en lecture seule.	Détection kernel/runtime Kubernetes à la place de Falco.
Sentinel	Gouvernance réseau : audit, planification et application contrôlée de NetworkPolicies.	Analyse comportementale des processus.
K-Guard AI	Enrichissement/contextualisation des événements runtime.	Remplacer Falco ou devenir une autorité de sécurité autonome.

3. System Overview & exploitation K3s

Le module System Overview fournit le premier niveau de lecture opérationnelle du cluster. Il permet d'identifier rapidement l'état des workloads avant de basculer vers une investigation de sécurité ou de réseau. La documentation actuelle indique notamment la visibilité des namespaces, pods, CPU/RAM, IP de pods, statut runtime, latence, version K3s et système hôte. ■ ■ ■

Capacités d'exploitation

- Inventaire des pods et deployments hors namespaces système.
- Lecture des logs récents d'un pod depuis l'API Kubernetes.
- Collecte des métriques CPU/RAM via l'API Metrics Server.
- Lecture de la capacité CPU/RAM du nœud.
- Lecture de l'espace disque sur les points de montage critiques.
- Accès aux informations de version K3s et de système hôte.
- Endpoints protégés par JWT pour les opérations d'administration.

Posture SRE

Le module ne cherche pas à remplacer une stack d'observabilité complète. Il fournit plutôt un point de contrôle opérationnel intégré au dashboard. Cette approche limite les allers-retours entre outils pour les diagnostics de premier niveau et garde les opérations plus sensibles dans le périmètre backend.

Maintenance

Une fonction de nettoyage contrôlé existe pour supprimer uniquement des fichiers temporaires approuvés. Le projet documente également les bonnes pratiques de maintenance et recommande de sauvegarder les données persistantes avant toute opération sensible. ■ ■ ■

4. Security Runtime — Falco & K-Guard AI

La chaîne runtime actuelle est explicitement séparée des fonctions Wazuh : **Falco** → **Fluent Bit** → **K-Guard AI** → **K-Guard Security Runtime**. Falco détecte les comportements suspects, Fluent Bit relaie les événements, puis K-Guard AI enrichit et contextualise les données avant leur présentation dans l'interface native.

■ ■ ■

Flux de traitement

Étape	Fonction	Donnée principale
1	Falco	Détection runtime et événement de sécurité.
2	Fluent Bit	Transport / forwarding des événements.
3	K-Guard AI	Enrichissement, analyse et contextualisation disponibles.
4	K-Guard backend	Validation, persistance et exposition API.
5	Security Runtime UI	Investigation opérationnelle et visualisation.

Persistance et exploitation

- Les événements runtime sont persistés dans SQLite afin de conserver un historique exploitable par l'interface.
- L'API expose les événements avec leur source, sévérité, règle, priorité, payload brut et état/enrichissement IA.
- La vue native centralise tendances, distributions, conteneurs fréquemment affectés et activité récente.
- La documentation actuelle précise qu'un dashboard Kibana externe n'est pas nécessaire pour l'usage normal de la sécurité runtime. ■ ■ ■

Limite importante

K-Guard AI est présenté comme une couche d'enrichissement et de contextualisation. Il ne doit pas être interprété comme une autorité de décision autonome ni comme un remplacement du moteur de détection Falco.

5. Endpoint & Compliance — Wazuh

L'intégration Wazuh a été introduite progressivement puis étendue en v1.6.0. La version actuelle expose l'inventaire des agents, les alertes et une vue de posture dans K-Guard. Le backend reste l'unique intermédiaire entre le navigateur et les services Wazuh. ■ ■ ■ ■

Architecture Wazuh

Source	Traitement	K-Guard
Wazuh Manager	Client backend authentifié, normalisation des agents.	Endpoint inventory.
Wazuh Indexer	Client backend pour la récupération des alertes et agrégations.	Security alerts / overview.
K-Guard Backend	Regroupe les données sans exposer les credentials.	API protégée JWT.
Frontend	Consomme uniquement les réponses K-Guard.	Views Endpoint & Compliance.

Données exposées

- Nombre total d'agents, agents actifs, déconnectés et jamais connectés.
- Nom, identifiant, IP, groupe, version, dernier keep-alive.
- Système d'exploitation, plateforme, version et architecture.
- Alertes Wazuh récentes et statistiques de sévérité.
- Vue de posture normalisée.

Lecture seule par conception

K-Guard ne permet pas depuis son dashboard d'enrôler un agent, de redémarrer un agent, de modifier des règles ou politiques Wazuh, ni d'exposer les credentials ou jetons au navigateur. Cette séparation est explicitement documentée. ■ ■ ■

TLS inter-services

La documentation de versionnement indique que la communication K-Guard → Wazuh Manager utilise une validation TLS stricte, des SAN DNS Kubernetes et un certificat CA monté comme Secret Kubernetes en lecture seule. Les credentials Wazuh et la CA sont ainsi conservés dans le périmètre backend/infrastructure.

■ ■ ■

6. Network Sentinel & gouvernance réseau

Network Sentinel est le module de gouvernance réseau de K-Guard. Son rôle est de visualiser la topologie, d'évaluer la posture de sécurité et d'encadrer l'application de NetworkPolicies. La version 1.7.0 marque une évolution importante : les politiques sont désormais manipulées par groupes logiques et peuvent être ciblées par namespace. ■ ■ ■

Groupes de politiques

Groupe	Finalité
security-exceptions	Exceptions de sécurité nécessaires au fonctionnement.
infra-allow	Accès nécessaires à l'infrastructure.
application-bridges	Flux entre composants applicatifs.
external-access	Flux vers des dépendances ou services externes autorisés.
namespace-baseline	Protection de base appliquée à un périmètre namespace.

Workflow sécurisé

- 1. Le backend génère un plan de durcissement en lecture seule.
- 2. L'interface présente les groupes, risques, volumes de politiques et périmètre concerné.
- 3. L'administrateur sélectionne explicitement les groupes et, si nécessaire, les namespaces.
- 4. Une confirmation explicite est requise avant toute modification.
- 5. Le playbook Ansible applique uniquement le périmètre demandé.
- 6. La suppression est limitée aux ressources portant l'identité de gestion **k-guard-sentinel**.

Audit de posture

Le moteur d'audit évalue notamment la couverture des NetworkPolicies, le durcissement des pods, l'exposition de services, l'activation de Pod Security Admission et certains signaux de Supply Chain. Le résultat fournit un score, une couverture, un niveau de confiance, des catégories et des findings. ■ ■ ■ ■

Validation v1.7.0

La release a été validée avec un plan Sentinel complet comprenant 47 politiques, 11 namespaces et 11 workloads. Des tests Ansible en check mode ont également été réalisés pour le durcissement groupé et la sélection des suppressions. ■ ■ ■

7. Sécurité applicative et modèle d'accès

K-Guard applique une architecture backend-mediated : les ressources protégées, les intégrations de sécurité et les credentials ne sont pas accessibles directement depuis le navigateur. Cette règle constitue un des principaux contrôles de réduction de surface d'exposition. ■ ■ ■

Authentification

- Authentification par OAuth2 Password Flow côté API.
- Mot de passe administrateur vérifié contre un hash bcrypt.
- Émission d'un JWT signé en HS256.
- Expiration configurée à 600 minutes dans l'implémentation actuelle.
- Les routes sensibles utilisent une dépendance de vérification JWT.

Protection des secrets

- Credentials Wazuh conservés côté backend / Secret Kubernetes.
- Token Cisco Webex manipulé côté backend.
- Clés JWT et mots de passe administrateur hors du frontend.
- Clés privées TLS exclues du dépôt et du périmètre documentaire public.
- Le projet documente explicitement l'interdiction de publier les secrets dans Git, captures, tickets ou messages.

■ ■ ■

Contrôle d'accès Kubernetes

K-Guard s'appuie sur l'API Kubernetes depuis le backend pour lire l'état du cluster et, dans le périmètre Sentinel, piloter les opérations de NetworkPolicy. La version 1.7.0 renforce le principe de moindre changement : groupes et namespaces doivent être explicitement sélectionnés et les suppressions sont limitées aux ressources gérées par Sentinel. ■ ■ ■

Point de vigilance

Le code actuel conserve des éléments qui doivent être durcis avant une exploitation de production, notamment la durée de vie du JWT, la gouvernance fine des permissions Kubernetes et l'exécution d'Ansible depuis le périmètre applicatif. Le code lui-même recommande une approche Operator ou déclenchement CI/CD dédié pour ce dernier point.

8. Intégrations et notifications

Cisco Webex fournit une boucle de notification opérationnelle. L'intégration est configurable depuis Settings, mais le token est conservé et utilisé par le backend. La documentation actuelle précise qu'il ne doit jamais être partagé publiquement. ■ ■ ■

Synchronisation Webex

- Configuration enregistrée dans SQLite.
- Token injecté dans un Secret Kubernetes dédié.
- Room ID synchronisé dans la configuration Fluent Bit.
- Redémarrage contrôlé du DaemonSet Fluent Bit pour prendre en compte la configuration.
- Les erreurs de notification ne doivent pas faire tomber le traitement principal.

API et intégrations

Intégration	Sens	Contrôle
Kubernetes API	K-Guard → K3s	Backend authentifié / opérations ciblées.
Wazuh Manager	K-Guard → Wazuh	Backend-only, TLS, lecture seule.
Wazuh Indexer	K-Guard → Indexer	Backend-only, lecture des alertes.
Falco / Fluent Bit	Security stack → K-Guard	Flux d'événements runtime.
K-Guard AI	Runtime pipeline → AI	Enrichissement/contextualisation.
Cisco Webex	K-Guard → Webex	Token côté backend, notification optionnelle.

9. Déploiement, observabilité et maintenance

K-Guard est déployé sur K3s et fournit une installation documentée destinée aux environnements Debian/Ubuntu Server 64-bit avec accès administrateur et connectivité réseau pendant l'installation. ■ ■ ■

Déploiement

- Le projet conserve un installateur Go pour automatiser la préparation et le déploiement.
- Le runtime applicatif repose sur FastAPI / Python et une interface Vue 3 / TypeScript.
- Les composants sont déployés dans le cluster et les intégrations sensibles restent côté backend.
- La documentation recommande des images Kubernetes immuables et documente le déploiement par SHA pour les releases. ■ ■ ■

Observabilité applicative

- Endpoint /health pour la liveness de l'application.
- Endpoint /api/health pour vérifier la disponibilité de l'API.
- Logs applicatifs et middleware de traçage des requêtes.
- Métriques Kubernetes consommées via Metrics Server pour l'interface.
- Diagnostic du stockage et lecture des logs de pods depuis le backend.

CI/CD et validation

Contrôle	Validation v1.7.0
Python	Compilation des modules Sentinel.
Frontend	Build Vue / TypeScript / Vite réussi.
Ansible	Syntaxe des playbooks de hardening et suppression validée.
Sentinel	Check mode et ciblage namespace testés.
Policy plan	47 policies / 11 namespaces / 11 workloads vérifiés.
Repository	Backups obsolètes, .bak, bytecode et artefacts générés retirés du suivi Git.

10. Validation v1.7.0 et limites

La release 1.7.0 a été publiée le 10 août 2026. Les changements principaux portent sur la gestion groupée des NetworkPolicies, le ciblage des namespaces, les plans de politique en lecture seule et le renforcement du cycle de vie des politiques Sentinel. ■ ■ ■

Ce qui est démontré

- Conception d'une application Cloud-Native orientée Kubernetes/K3s.
- Intégration de plusieurs sources de sécurité avec séparation de responsabilités.
- Backend sécurisé comme intermédiaire entre UI et services protégés.
- Gestion de NetworkPolicies avec périmètre explicite et confirmation utilisateur.
- Automatisation Ansible et validation en check mode.
- Exposition native des informations runtime, endpoint et réseau.
- Documentation et versionnement du projet par releases.

Ce qui n'est pas revendiqué

- K-Guard ne remplace pas un SOC complet ni un processus de réponse à incident.
- La vue Wazuh actuelle ne revendique pas encore la disponibilité des index SCA/vulnérabilités ; l'API indique explicitement que ces données ne sont pas encore disponibles. ■ ■ ■
- Le projet n'est pas présenté comme ayant subi un audit de sécurité indépendant.
- Les mécanismes d'auto-remédiation agressive de l'ancienne architecture Trivy ne font pas partie du périmètre documentaire v1.7.0.
- Une validation de production doit encore couvrir les permissions Kubernetes, la gestion des secrets, les scénarios de panne et les procédures de reprise.

Évolution depuis les versions précédentes

La v1.7.0 doit être comprise comme une étape de maturation : l'architecture historique de v1.4 comportait encore des mécanismes Trivy et Smart Patching qui ne représentent plus le cœur de la version actuelle. Cette documentation remplace donc l'ancien dossier comme référence de la release actuelle, tout en laissant l'ancienne version comme archive historique.

11. Conclusion et perspectives

K-Guard v1.7.0 constitue un laboratoire Cloud-Native orienté sécurité opérationnelle et exploitation K3s. La valeur du projet ne repose pas uniquement sur l'interface : elle réside dans la mise en relation contrôlée de Kubernetes, Falco, K-Guard AI, Wazuh, Ansible, NetworkPolicies et Cisco Webex derrière un backend sécurisé.

L'évolution récente montre un déplacement du projet vers une gouvernance plus explicite : moins d'actions globales implicites, davantage de plans en lecture seule, de sélection de périmètre, de confirmation et de traçabilité. Cette évolution est particulièrement visible dans Sentinel v1.7.0. ■ ■ ■

Perspectives techniques

- Renforcer le modèle RBAC Kubernetes et réduire davantage les permissions du backend.
- Externaliser l'exécution Ansible hors du pod applicatif, par exemple via un mécanisme CI/CD ou un opérateur dédié.
- Poursuivre la maturation de K-Guard AI avec des critères d'enrichissement et de confiance explicites.
- Étendre la visibilité Wazuh lorsque les données SCA/vulnérabilités seront réellement disponibles.
- Améliorer la couverture de tests automatisés et les scénarios de défaillance.
- Renforcer la gestion des secrets et la rotation des credentials.
- Documenter les procédures de sauvegarde, restauration et reprise après incident.

Posture professionnelle

Le projet est volontairement présenté avec ses forces et ses limites. L'objectif est de démontrer une capacité à concevoir, sécuriser, documenter et faire évoluer une infrastructure logicielle réelle, tout en conservant une posture réaliste sur ce qui reste à durcir avant un usage de production.

K-Guard v1.7.0

Security by Design • Kubernetes/K3s • Runtime Security • SIEM/XDR • Network Governance • Automation